Predicting Fatal vs. Non-Fatal Gun Violence Incidents from Structured Incident-Level Features

Jiahao Zhang (jiahaoz@uchicago.edu)

2026-03-11

1. Introduction

This report studies a practical prediction problem: using structured incident-level information, can we distinguish gun violence incidents that become fatal from those that do not? The outcome is binary, fatal versus non-fatal, and the predictors are limited to structured variables that are plausibly available from the incident record itself, including time, broad location, district identifiers, basic gun information, and simple derived indicators from the address and age-group fields. The practical value is not perfect classification, but whether structured incident records can support earlier screening of higher-risk cases. The goal is predictive rather than causal. I am not estimating the effect of any one factor. I am asking whether a cleaned, defensible feature set contains useful signal for classification. More substantively, the project also asks whether the limited structured information available in incident records is enough to support meaningful severity screening, and whether the usable signal comes mainly from immediate incident descriptors or from broader contextual and place-related markers.

Recent work on firearm violence and related public-safety outcomes suggests two relevant points. First, even noisy administrative or surveillance-style data can contain predictive information about victimization or severe injury outcomes (Goin, Rudolph, and Ahern 2018; Heller et al. 2022; Polcari et al. 2023; Zhou et al. 2024). Second, prediction is still difficult because these data are incomplete, unevenly reported, and often dominated by non-event or lower-severity cases (Swedo et al. 2023). Prior work also emphasizes that the practical goal is often screening or community risk identification rather than perfect classification (Polcari et al. 2023). Those concerns matter directly here. Structured contextual variables may still carry signal, but missingness can distort what is usable, and class imbalance can make a model look stronger than it really is if evaluation relies too heavily on raw accuracy.

This project gives a clean comparison. I keep the revised leakage-safe pipeline, use logistic regression as the formal baseline model, compare it against two selected nonlinear models on

the same final feature set, and then examine how class weighting, oversampling, and threshold moving change the practical detection of fatal incidents. That second step matters because the model with the best overall ranking is not automatically the best decision rule for the minority class.

Repository / shared materials link: macss30100_Gun_violence

2. Data and Outcome Definition

The original records come from Gun Violence Archive incident listings that had already been assembled for the course project into a local structured file. The analysis in this report begins from `stage3.csv`, which is the project's cleaned stage-three working dataset rather than a live scrape. It contains incident-level observations, original URLs, and a larger set of structured fields created earlier in the project workflow. The underlying source records were collected earlier in the course project workflow, and the present report does not re-scrape the live source. Instead, it starts from the saved cleaned file `stage3.csv`, which was generated from those earlier source records and retained as the local analysis dataset for the final report. In other words, `stage3.csv` is the final local analysis file derived from the original incident-source records, and every table and figure below is produced from that file or from saved result tables generated from it.

The binary outcome is defined as

```
stage3 <- stage3 |>
  mutate(
    fatal = factor(
      if_else(n_killed > 0, "fatal", "non_fatal"),
      levels = c("non_fatal", "fatal")
    )
  )
```

[$\texttt{fatal} = 1(\texttt{n_killed} > 0)$]

This definition is simple, auditable, and aligned with the question of whether an incident crossed the line from non-fatal to fatal harm. I use this binary target instead of directly modeling `n_killed` because the death count is highly right-skewed with many zeros, so a count outcome would mix a much harder distributional problem into what is mainly a screening task. It also has limits. It does not distinguish one death from multiple deaths, and it treats all non-fatal incidents as one group even when injury severity varies a lot. So the target is useful for a first predictive exercise, but it is still a coarse measure of harm.

Table 1: Working dataset and outcome overview

metric	value
Rows in stage3.csv	239,677
Columns in stage3.csv	32
Date range	2013-01-01 to 2018-03-31
Fatal incidents	53,842
Non-fatal incidents	185,835
Fatal share	22.5%

The fatal class accounts for about 22.5% of the full file. That imbalance is important from the start because a model can achieve respectable accuracy while still missing most fatal incidents. For that reason, later sections emphasize balanced accuracy and fatal recall rather than accuracy alone.

3. Exploratory Data Analysis

The EDA is used here to support modeling choices rather than to survey every field in the file. I focus on dataset coverage, predictor structure, and a few patterns that motivate the final retained feature set.

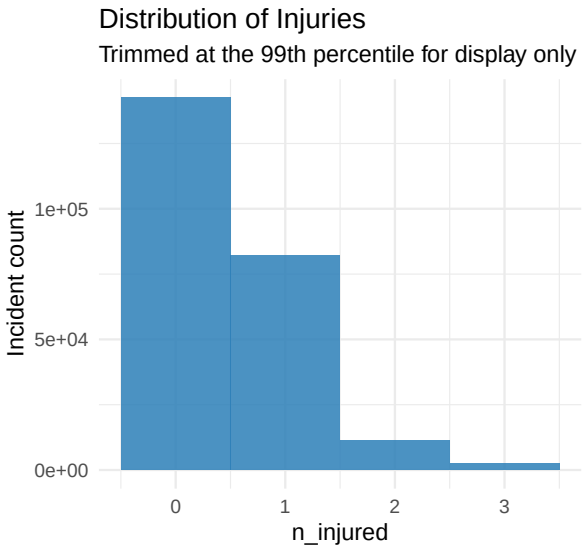
3.1 Dataset Overview

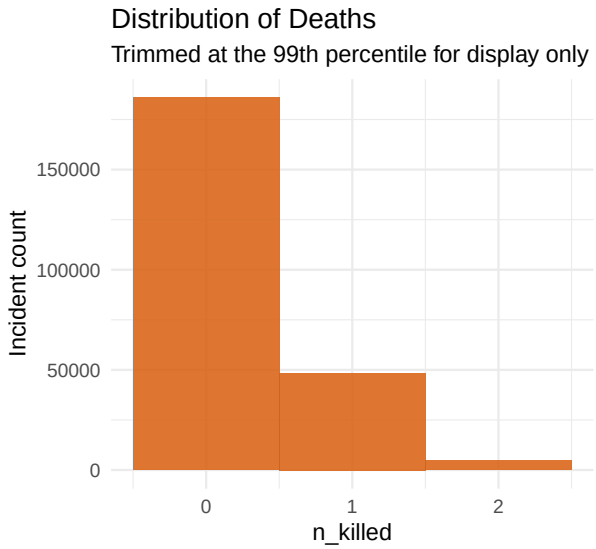
Table 2: Top 15 fields by missingness

feature	missing_n	missing_pct
participant_relationship	223903	93.4%
location_description	197588	82.4%
participant_name	122253	51.0%
gun_stolen	99498	41.5%
gun_type	99451	41.5%
n_guns_involved	99451	41.5%
participant_age	92298	38.5%
notes	81017	33.8%
participant_age_group	42119	17.6%
state_house_district	38772	16.2%
participant_gender	36362	15.2%
state_senate_district	32335	13.5%
participant_status	27626	11.5%

feature	missing_n	missing_pct
participant_type	24863	10.4%
address	16497	6.9%

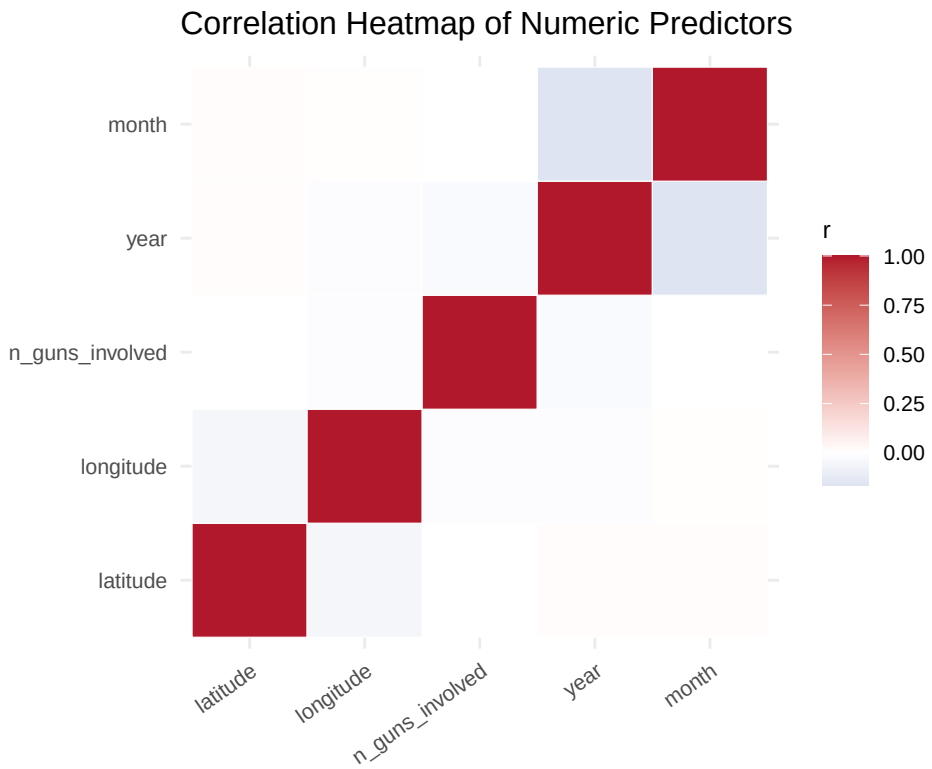
Several detailed relationship, participant, and narrative-style variables have substantial missingness. That matters because sparse fields can create an illusion of richness without producing stable predictive signal. It also supports a conservative modeling decision: use the cleaner structured variables and avoid heavily reported-after-the-fact fields in the final predictive set.



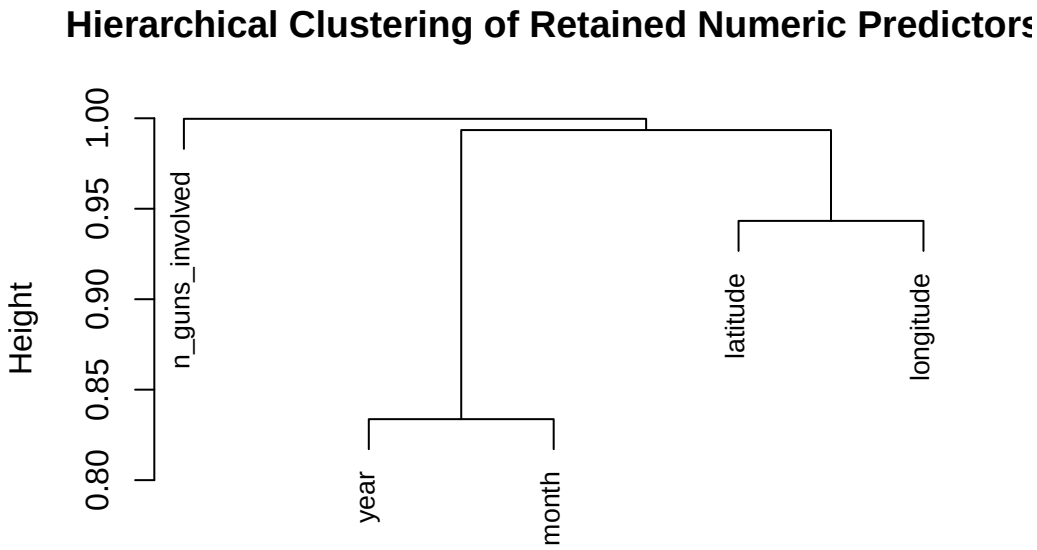


The casualty distributions are strongly right-skewed. Most incidents have low counts, while a smaller number of extreme events create a long tail. This shows that the data are dominated by lower-casualty incidents, so a simple average would hide the basic shape of the outcome. It also helps explain why the report focuses on a binary fatal indicator instead of directly modeling the count of deaths. A count model would have to deal with both heavy skew and a large concentration of small values, while the present project is more directly interested in whether an incident becomes fatal at all.

3.2 Predictor Overview

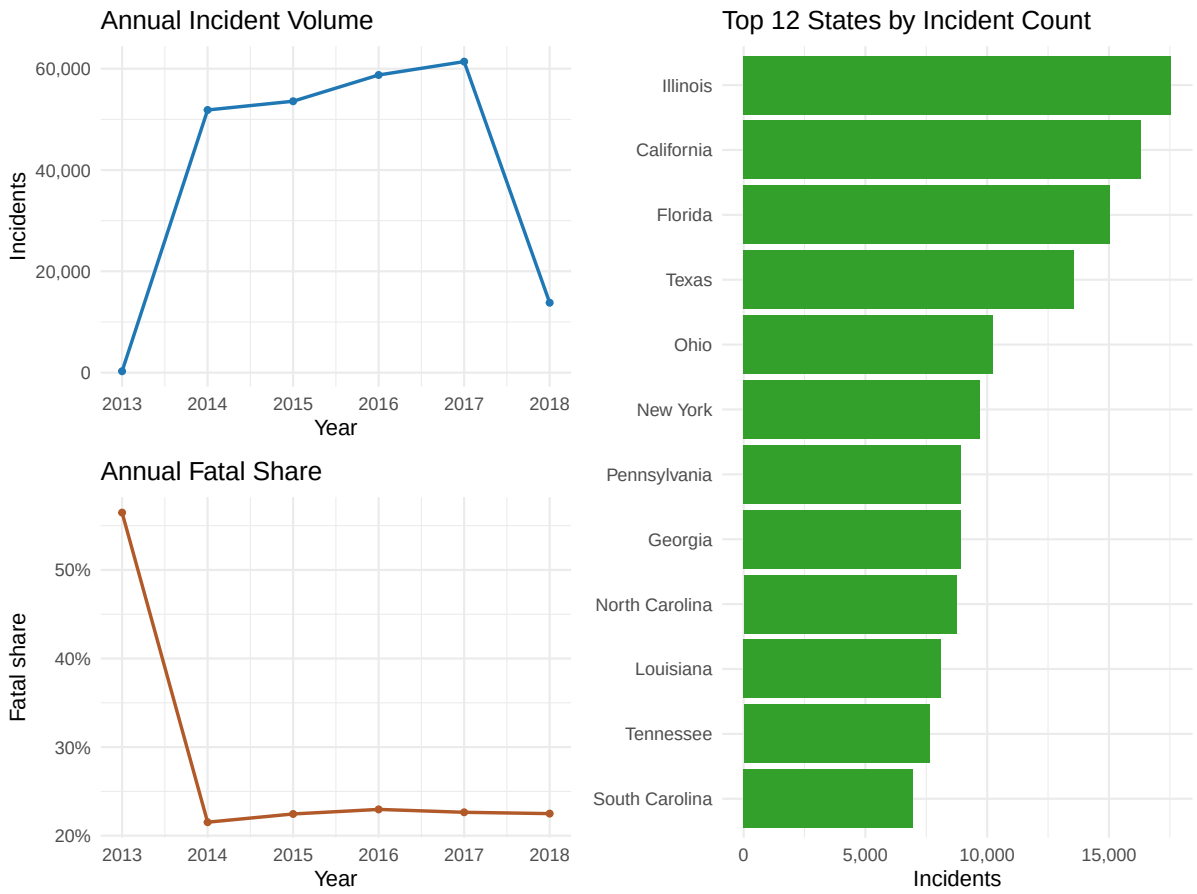


The numeric predictors that remain in the final operational feature set are not strongly collinear. This is helpful because the model is not depending on a single duplicated numeric signal, and it reduces the risk that the final classifier appears more stable than it really is because several variables are carrying nearly the same information. At the same time, the district identifiers are not treated as numeric quantities in modeling. They are identifiers, so they are handled as categorical variables later in the pipeline rather than being interpreted as ordered magnitudes.

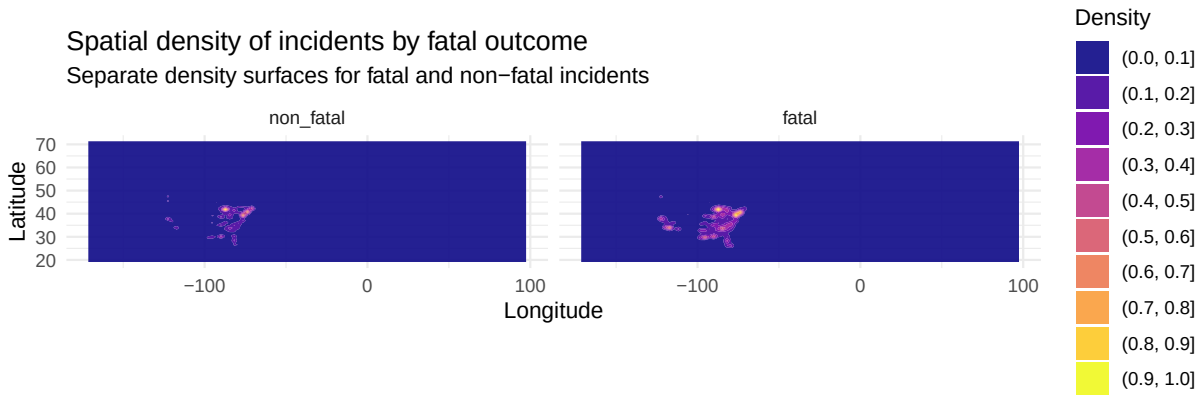


The clustering result is consistent with the heatmap: there is no tight block of highly redundant numeric features in the retained set. Latitude and longitude naturally sit closer to each other than to the time or gun-count variables, but the overall structure remains fairly weak. In practical terms, this means the retained numeric predictors seem to contribute different kinds of information rather than collapsing into one dominant dimension. That pattern supports keeping these predictors together in the final pipeline and treating each as a separate possible source of signal.

3.3 Initial Patterns

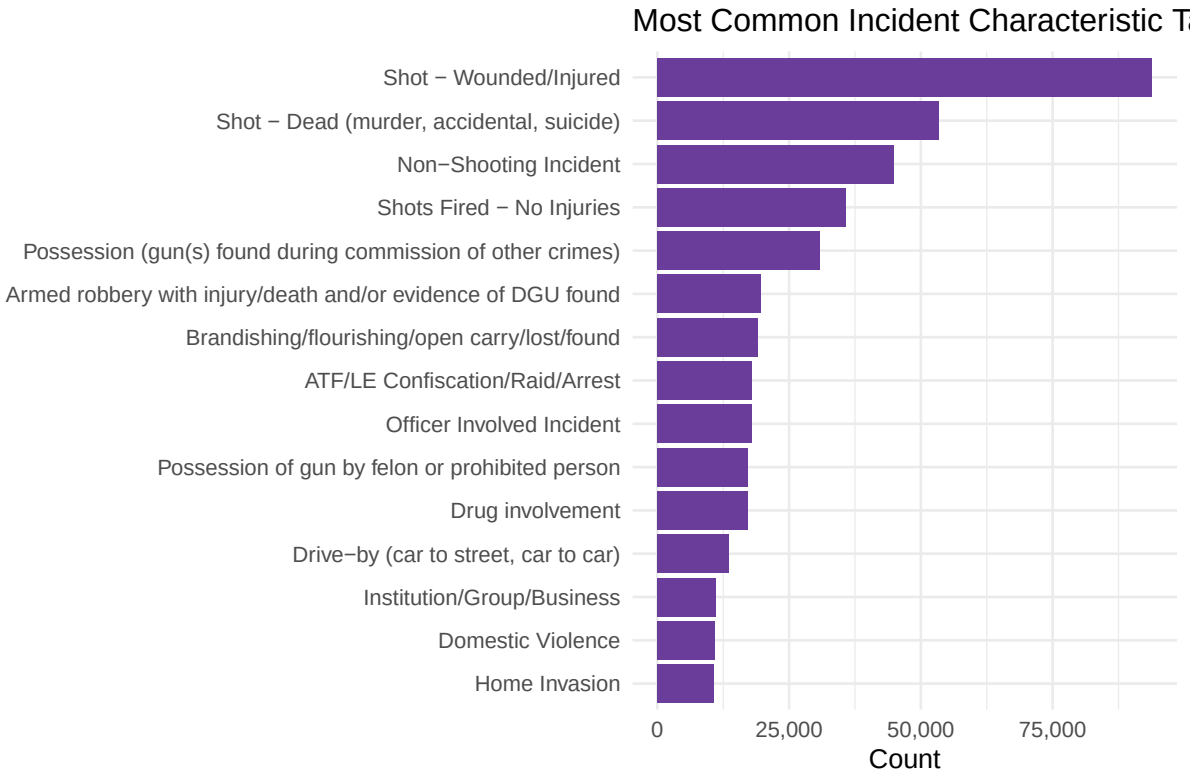


Incident volume and fatal share do not move in the same way across years or across states. Some places contribute many incidents without having the highest fatal share, while the year-to-year pattern in incidence does not mechanically track the year-to-year pattern in fatality. This supports keeping time and location variables in the predictive set, because they appear to carry nontrivial contextual information. At the same time, it suggests that those fields should be interpreted as markers of place and timing rather than as direct explanations of why an individual incident becomes fatal.



This figure gives a more direct spatial view of the same point. Incidents are not spread evenly across the map, and the highest-density areas for fatal and non-fatal incidents overlap only partly rather than perfectly. That does not mean location alone determines fatality, but it does suggest that geographic context carries predictive information beyond simple incident counts. These spatial differences may reflect contextual and reporting differences, not causal effects of place itself. It also helps explain why location variables later appear as important contributors in the tree-based models. Overall, the figure supports keeping latitude, longitude, and broader administrative location fields in the final feature set while still treating them as contextual predictors rather than causal claims.

For presentation in the PDF report, I keep this static density figure because it is easier to read on the page. I also created an interactive leaflet map that extends this same spatial view and makes it easier to inspect local clustering more directly. The interactive version can be accessed through the GitHub repository link listed in the Introduction, under the project materials and README, where it is linked as an interactive map webpage. The static density figure above remains the clearer choice for the main written report.



I keep `incident_characteristics` in the EDA because it is descriptive and helps show what kinds of incidents appear in the file. It gives a clearer sense of the substantive mix of events represented in the dataset and helps explain why the source file is tempting to over-engineer. At the same time, I do not use it in the final model because it is a documentation-heavy field. Using this field would likely improve apparent predictive power, but at the cost of mixing incident information with after-the-fact coding intensity. More detailed tagging may reflect how fully an incident was later described or coded, including local reporting habits, rather than the incident itself. In other words, it is useful for understanding the dataset, but too risky to treat as a clean predictor.

4. Feature Construction and Final Modeling Set

The final modeling set is intentionally narrower than the full file. I keep only structured fields that are reasonably interpretable as incident-time or near-incident information and that can be represented cleanly in a predictive pipeline. In practice, the retained predictors fall into five groups: time markers (`year`, `month`, `day_of_week`, `weekend`), broad location fields (`state`, `city_or_county`, latitude, longitude), administrative district identifiers (`congressional_district`, `state_house_district`, `state_senate_district`), basic gun-related variables (`gun_stolen`, `gun_type`, `n_guns_involved`), and simple engi-

neered indicators from existing structured text (`addr_has_block`, `addr_has_intersection`, `addr_has_road`, `has_teen`, `has_child`).

I keep these variables for straightforward reasons. Time and broad location may capture recurring seasonal or geographic context. The district fields may absorb coarse administrative or neighborhood patterning, but I treat them only as identifiers, not as ordered numeric quantities. The gun variables are among the few structured incident descriptors available in the file. The address flags and age-group indicators are simple summaries that can be built from the existing record without introducing long narrative text into the model.

I exclude variables that are either too close to the outcome or too vulnerable to leakage and documentation effects. In particular, I do not use `n_injured`, because it is too tightly connected to incident severity and would blur the distinction between predictors and consequences. I also do not restore `notes`, `location_description`, or `incident_characteristics`, because richer descriptions may reflect reporting intensity, local documentation habits, or after-the-fact write-up rather than characteristics of the incident itself. This same logic is why I do not add back summary features derived from those narrative-style fields. The goal here is not to maximize predictive power at any cost, but to keep a defensible final feature set.

Missing data are handled conservatively rather than by dropping incomplete rows. The main missingness problem in the final retained set is in a few numeric and location-related fields, especially `n_guns_involved`, `latitude`, and `longitude`, while many categorical fields also contain explicit unknown-style values or sparse rare levels. I keep the rows and handle those issues inside the training-only preprocessing pipeline. Numeric predictors receive median imputation plus missingness indicators, so the model can use both the imputed value and the fact that the value was originally missing. Categorical predictors receive an explicit unknown level, and very rare levels in high-cardinality fields are pooled into `"other"` before dummy encoding. That approach keeps the sample size stable and avoids leaking information from the full dataset into the training step.

```
feature_snippet <- stage3 |>
  transmute(
    fatal,
    year = factor(year),
    month = factor(month),
    state = factor(state),
    city_or_county = factor(city_or_county),
    gun_stolen = factor(gun_stolen),
    gun_type = factor(gun_type),
    n_guns_involved,
    latitude,
    longitude,
    congressional_district = factor(congressional_district),
    addr_has_block = factor(if_else(str_detect(replace_na(address, ""), regex("block", ignore
```

```
    has_teen = factor(if_else(str_detect(replace_na(participant_age_group, ""), stringr::fixe
  )
```

This short excerpt is only a representative view of the full feature construction. It shows the main pattern: structured timing and location fields are retained directly, while a small number of simple indicators are derived from existing text fields without bringing narrative content into the final model.

Table 3: Final retained modeling features

feature	type	missing_pct	distinct_values
year	categorical	0.0%	6
month	categorical	0.0%	12
day_of_week	categorical	0.0%	7
weekend	categorical	0.0%	2
state	categorical	0.0%	51
city_or_county	categorical	0.0%	12821
gun_stolen	categorical	41.5%	349
gun_type	categorical	41.5%	2502
n_guns_involved	numeric	41.5%	106
latitude	numeric	3.3%	101240
longitude	numeric	3.3%	112347
congressional_district	categorical	5.0%	54
state_house_district	categorical	16.2%	275
state_senate_district	categorical	13.5%	68
addr_has_block	categorical	0.0%	2
addr_has_intersection	categorical	0.0%	2
addr_has_road	categorical	0.0%	2
has_teen	categorical	0.0%	2
has_child	categorical	0.0%	2

To make the retained features more concrete, the next two tables summarize the numeric and categorical predictors separately. The most important point is that the retained raw feature set is relatively compact and interpretable before dummy expansion: three numeric variables, several time and location identifiers, three district identifiers treated as factors, three simple address flags, and two participant age-group indicators.

Table 4: Descriptive statistics for retained numeric features

feature	missing_pct	mean	sd	median	p90
latitude	3.3%	37.55	5.13	38.57	42.81
longitude	3.3%	-89.34	14.36	-86.25	-75.11
n_guns_involved	41.5%	1.37	4.68	1.00	1.00

Table 5: Summary of retained categorical features

feature	missing_pct	distinct_values	mode	mode_share
addr_has_block	0.0%	2	no	65.3%
addr_has_intersection	0.0%	2	no	85.3%
addr_has_road	0.0%	2	yes	66.3%
city_or_county	0.0%	12821	Chicago	4.5%
congressional_district	5.0%	54	1	16.2%
day_of_week	0.0%	7	Sunday	15.5%
gun_stolen	41.5%	349	0::Unknown	86.5%
gun_type	41.5%	2502	0::Unknown	66.7%
has_child	0.0%	2	no	98.5%
has_teen	0.0%	2	no	91.9%
month	0.0%	12	1	9.6%
state	0.0%	51	Illinois	7.3%
state_house_district	16.2%	275	18	1.7%
state_senate_district	13.5%	68	5	4.8%
weekend	0.0%	2	weekday	69.5%
year	0.0%	6	2017	25.6%

Taken together, these tables answer two practical questions. First, what are the features? They are the structured time, location, gun, district, address, and age-group variables listed above and in Table 4. Second, what do they look like in the data? The numeric summary shows scale and missingness for the continuous predictors, while the categorical summary shows how much variation each factor contains and whether it is dominated by one common level. I still retain `city_or_county` because the EDA and later tree-based interpretation both suggest that broad geographic context carries predictive signal, even if that signal must be interpreted cautiously.

For the comparative modeling stage, I target a stratified 40,000-observation analysis sample drawn after the initial train/test split. This enlarged sample strengthens the main comparison among random forest, XGBoost, and logistic regression without changing the cleaned feature logic. The only smaller sample used in the report itself is the SHAP interpretation subset,

which is used only for computational convenience after the preferred model has already been fit.

5. Modeling Strategy

5.1 Clean Pipeline

The key modeling principle is that the split comes before all learned preprocessing. Missing-value handling, rare-level consolidation, dummy encoding, and normalization are estimated on the training data only and then applied to validation or test data. That is the main correction that prevents the leakage problem from the earlier draft.

```
split_obj <- initial_split(analysis_df, prop = 0.80, strata = fatal)
train_data <- training(split_obj)
test_data <- testing(split_obj)

model_n <- min(40000, nrow(analysis_df))
train_n <- floor(model_n * 0.80)
test_n <- model_n - train_n

if (nrow(train_data) > train_n) {
  train_data <- train_data |>
    group_by(fatal) |>
    slice_sample(prop = train_n / nrow(train_data)) |>
    ungroup()
}

if (nrow(test_data) > test_n) {
  test_data <- test_data |>
    group_by(fatal) |>
    slice_sample(prop = test_n / nrow(test_data)) |>
    ungroup()
}

basic_recipe <- recipe(fatal ~ ., data = train_data) |>
  step_indicate_na(n_guns_involved, latitude, longitude) |>
  step_impute_median(all_numeric_predictors()) |>
  step_unknown(all_nominal_predictors()) |>
  step_other(city_or_county, gun_stolen, gun_type, threshold = 0.01, other = "other") |>
  step_novel(all_nominal_predictors()) |>
  step_dummy(all_nominal_predictors()) |>
  step_zv(all_predictors())
```

Table 6: Train/test split diagnostics for the 40,000-observation modeling sample

split	n	fatal_share
train	31999	22.5%
test	7999	22.5%

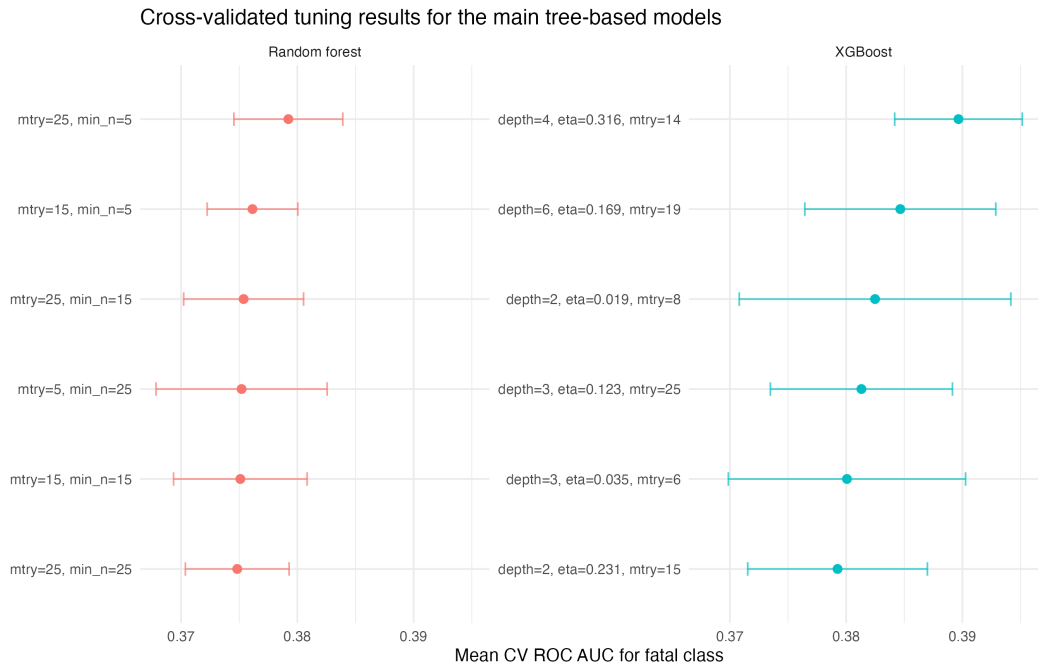
This preprocessing corresponds directly to the feature decisions in Section 4. `step_indicate_na()` preserves information about whether key numeric variables were originally missing. `step_impute_median()` fills those numeric gaps without borrowing from the test set. `step_unknown()` and `step_novel()` make categorical handling more stable when some levels are absent in one split but appear in another. `step_other()` is especially important for `city_or_county`, `gun_stolen`, and `gun_type`, because those fields contain many sparse categories that would otherwise create a very wide and unstable design matrix. Finally, `step_dummy()` converts the retained categorical features into model-ready indicators. For scale-sensitive models, I then extend this shared `basic_recipe` into a separate `normalized_recipe` by adding `step_normalize(all_numeric_predictors())`; that extra step is used for logistic regression in the comparison workflows, while the tree-based models keep the unnormalized version.

5.2 Models Compared

The baseline comparison keeps three primary classifiers: logistic regression, random forest, and XGBoost. Logistic regression is the formal baseline because it is the simplest and most transparent benchmark. Random forest and XGBoost are the two selected additional models because they can capture nonlinear structure and interactions in structured tabular data.

For the main model-selection stage, I tuned the tree-based models inside the training data only. Random forest was tuned over `mtry` and `min_n` with `trees = 400` fixed. XGBoost was tuned over `trees`, `tree_depth`, `learn_rate`, `mtry`, `min_n`, `loss_reduction`, and `sample_size`. In both cases, I used cross-validation and selected the final specification by ROC AUC for the fatal class, entirely within the training and resampling framework rather than on the held-out test set. The baseline comparison therefore reflects the formal logistic baseline alongside tuned nonlinear comparison models. After those hyperparameters were fixed, I kept the resulting model specifications stable in the later imbalance analysis so that the report would focus on decision rules and classification trade-offs rather than repeated retuning.

The next figure is a tuning summary rather than a final-results figure. Each point represents one candidate hyperparameter setting, and the horizontal position shows its mean cross-validated ROC AUC for the fatal class. The purpose of this step is to show that the tree-based models were tuned inside the training data before the final held-out comparison.



The tuning results show that the better random-forest settings were concentrated in a relatively small part of the search grid, while XGBoost was more sensitive to the joint choice of depth, learning rate, and feature subsampling. I do not treat the exact winning values as substantively meaningful, because small differences across candidate settings can easily reflect finite-sample variation rather than a deep substantive truth about the problem. The point of the tuning step is simply to avoid comparing a reasonably tuned tree model against a poorly specified one. It also helps show that the later baseline comparison is not driven by arbitrary defaults alone. After the tuning stage, I carry the selected random-forest specification into the main comparison with the workflow below.

```
rf_fit <- workflow() |>
  add_recipe(basic_recipe) |>
  add_model(rf_spec) |>
  fit(train_data)
```

5.3 Evaluation Metrics

I report ROC AUC, accuracy, balanced accuracy, fatal sensitivity, and non-fatal specificity. ROC AUC describes how well a model ranks fatal incidents above non-fatal incidents across all possible thresholds. Accuracy is the overall share of correct classifications, but in this dataset it can be misleading because non-fatal incidents are much more common. Sensitivity for the fatal class means the share of truly fatal incidents that the model correctly flags as

fatal. Specificity for the non-fatal class means the share of truly non-fatal incidents that the model correctly leaves in the non-fatal category. Balanced accuracy is the average of those two rates, so it is useful here because it does not let the majority class dominate the summary.

In this setting, a model with low sensitivity misses many fatal incidents, while a model with low specificity falsely flags many non-fatal incidents as fatal. That is why the later imbalance analysis emphasizes the trade-off between these two quantities rather than reading accuracy alone.

The metric rule in the rest of the report has two steps. First, I choose the underlying classifier by ROC AUC because that metric evaluates ranking performance without fixing a cutoff. Second, after the underlying classifier is chosen, I compare operating thresholds. At that stage, balanced accuracy is the main summary because it keeps the minority and majority classes in view at the same time. But I do not treat it mechanically by itself. For this application, fatal recall still matters more than precision or raw accuracy alone, because missing a truly fatal incident is more costly than flagging some non-fatal incidents as higher risk. At the same time, I do not want false positives to grow without bound, so non-fatal specificity also matters. In other words, ROC AUC chooses the underlying classifier, while balanced accuracy helps compare thresholds, and recall plus specificity determine whether a threshold is practically acceptable for the screening-style objective. For the final operating rule, I therefore compare balanced-accuracy-based threshold moving with a more conservative specificity-constrained threshold check.

6. Baseline Model Comparison

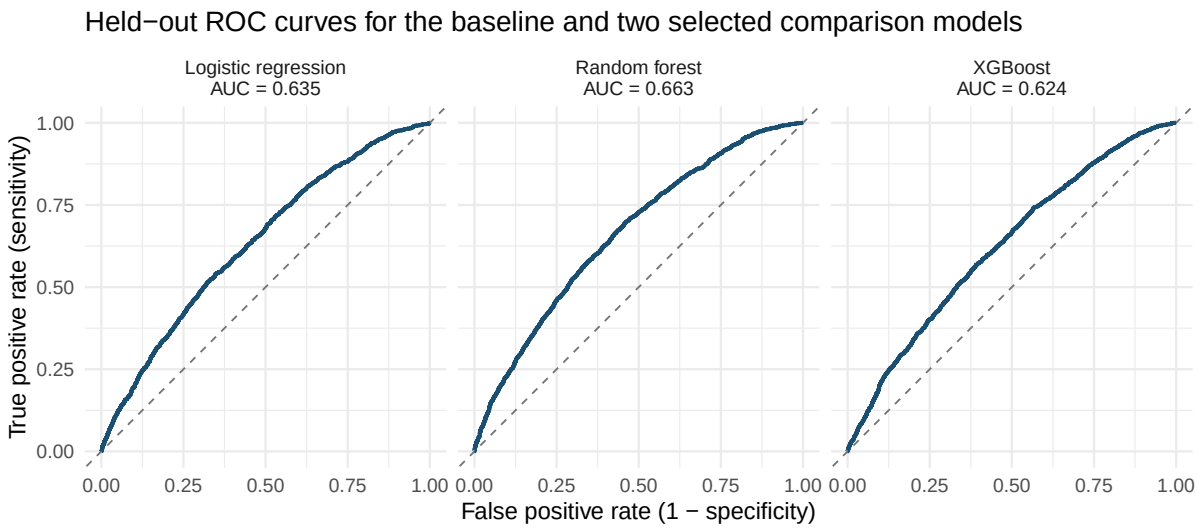
The baseline comparison uses the saved held-out results from `three_model_default_results.csv`, which apply the same cleaned feature set and default threshold of 0.50 on the 40,000-observation modeling sample.

Table 7: Default three-model comparison on the held-out test set

model	roc_auc	accuracy	bal_accuracy	sensitivity_fatal	specificity_nonfatal
Random forest	0.6632	0.7738	0.5233	0.0684	0.9782
Logistic regression	0.6348	0.7721	0.5165	0.0523	0.9807
XGBoost	0.6234	0.7753	0.5000	0.0000	1.0000

```
roc_curve_tbl |>
  left_join(roc_auc_labels, by = "model") |>
  ggplot(aes(x = 1 - specificity, y = sensitivity)) +
  geom_abline(linetype = 2, color = "gray45", linewidth = 0.4) +
```

```r
  geom_path(color = "#1B4F72", linewidth = 0.9) +
  facet_wrap(~ panel_label, ncol = 3) +
  coord_equal() +
  labs(
    title = "Held-out ROC curves for the baseline and two selected comparison models",
    x = "False positive rate (1 - specificity)",
    y = "True positive rate (sensitivity)"
  )
```



The ROC panels make the three model-ranking patterns easier to compare without forcing nearly overlapping curves into a single plot. The main takeaway is not that one model dominates every part of the curve, but that random forest keeps a somewhat stronger ranking profile across the threshold range than the other two primary models. Logistic regression serves as the formal baseline, but in the enlarged-sample comparison random forest is the strongest overall performer because it has the highest ROC AUC (0.6632) and the highest default balanced accuracy among the three primary models. This does not mean it already solves the practical classification problem under imbalance. Rather, it means it provides the strongest underlying score ranking before later decision-rule adjustments. I therefore carry the random forest forward as the preferred underlying classifier for the later imbalance adjustments. Despite moderate ROC AUC differences, all three default-threshold classifiers perform poorly at fatal-case detection.

7. Further Analysis: Class Imbalance and Decision Rules

The baseline results show why class imbalance matters. Several models have similar accuracy because they favor the majority class. To study this more directly, I compare three follow-up

strategies: class weighting, oversampling, and threshold moving. In each case, the held-out test set remains untouched. The goal of this section is not to maximize fatal recall at any cost, but to examine which adjustment gives the most reasonable compromise once fatal-case detection is treated as somewhat more important than avoiding every false alarm.

Table 8: Held-out comparison across imbalance-handling strategies

setup	threshold	roc_auc	accuracy	bal_accuracy	sensitivity_fatal	specificity_nonfatal
Random forest | default	0.50	0.6632	0.7738	0.5233	0.0684	0.9782
Random forest | class weight	0.50	0.6654	0.7732	0.5231	0.0690	0.9773
Random forest | oversampled	0.50	0.6572	0.7298	0.5756	0.2955	0.8557
Random forest | threshold moved	0.19	0.6632	0.5587	0.6127	0.7106	0.5147
XGBoost | default	0.50	0.6234	0.7753	0.5000	0.0000	1.0000
XGBoost | class weight	0.50	0.6202	0.7753	0.5000	0.0000	1.0000
XGBoost | oversampled	0.50	0.6266	0.5739	0.5901	0.6194	0.5608
XGBoost | threshold moved	0.23	0.6234	0.5986	0.5896	0.5732	0.6059
Logistic regression | default	0.50	0.6348	0.7721	0.5165	0.0523	0.9807
Logistic regression | oversampled	0.50	0.6318	0.5862	0.5901	0.5971	0.5830
Logistic regression | threshold moved	0.20	0.6348	0.5324	0.5910	0.6973	0.4847

7.1 Class Weighting

Class weighting keeps the original training rows but tells the algorithm to penalize mistakes on fatal incidents more heavily than mistakes on non-fatal incidents. The idea is to make the minority class more influential without duplicating observations.

```
class_ratio <- sum(train_data$fatal == "non_fatal") / sum(train_data$fatal == "fatal")

rf_weighted_fit <- workflow() |>
  add_recipe(basic_recipe) |>
  add_model(
    rand_forest(mtry = 25, min_n = 5, trees = 400) |>
      set_engine(
        "ranger",
```

```
        probability = TRUE,
        class.weights = c(non_fatal = 1, fatal = class_ratio)
      ) |>
      set_mode("classification")
  ) |>
  fit(train_data)
```

In this project, weighting changes little. For random forest, balanced accuracy moves only from 0.5233 under the default setup to 0.5231 after weighting. That is too small to change the practical conclusion. The same general pattern holds for XGBoost: weighting by itself does not solve minority-class detection.

The interpretation is that weighting does not move the fitted decision boundary very much in this dataset. The models still behave mostly like majority-class classifiers at the default cutoff, so the gain in fatal detection is minimal. For that reason, class weighting is the weakest of the three imbalance strategies examined here. It is methodologically clean and easy to justify, but in this project it does not produce enough practical improvement to be the preferred adjustment.

7.2 Oversampling

Oversampling changes the training distribution itself. Instead of reweighting errors, it resamples the minority class so that the fitted model sees a more balanced mix of fatal and non-fatal cases during training. In the final workflow I implement this with `themis::step_upsample()` inside the recipe, using `over_ratio = 1` so that the minority `fatal` class is expanded to a 1:1 ratio with the majority class in the training data. I place the step near the front of the recipe so that later preprocessing steps are learned from the balanced training sample, and I keep the held-out test set untouched.

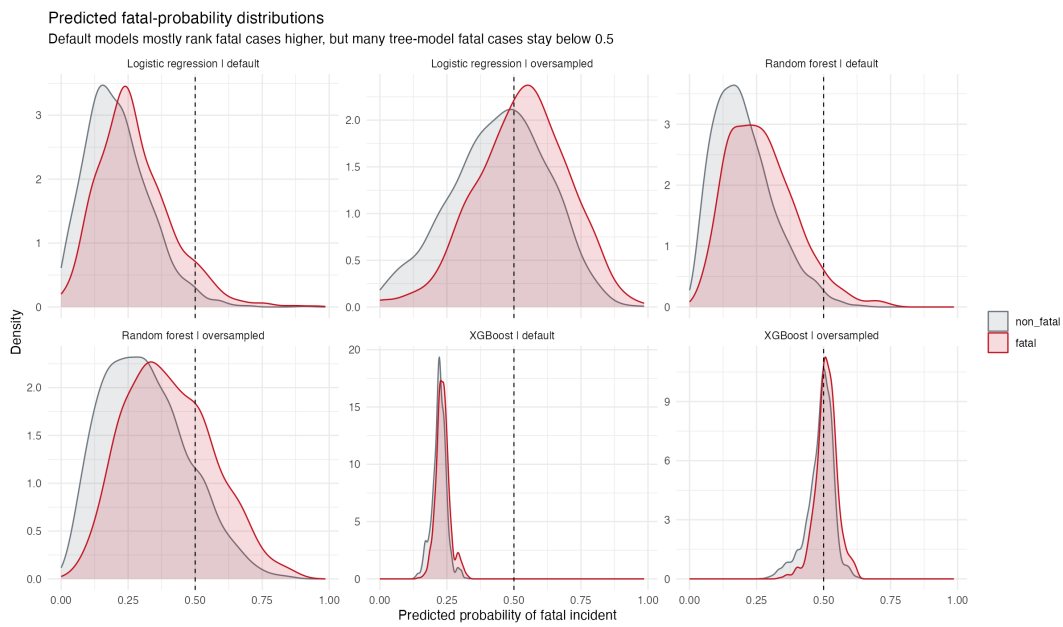
```
rf_oversampled_recipe <- recipe(fatal ~ ., data = train_data) |>
  step_upsample(fatal, over_ratio = 1, seed = 30100) |>
  step_indicate_na(n_guns_involved, latitude, longitude) |>
  step_impute_median(all_numeric_predictors()) |>
  step_unknown(all_nominal_predictors()) |>
  step_other(city_or_county, gun_stolen, gun_type, threshold = 0.01, other = "other") |>
  step_novel(all_nominal_predictors()) |>
  step_dummy(all_nominal_predictors()) |>
  step_zv(all_predictors())

rf_oversampled_fit <- workflow() |>
  add_recipe(rf_oversampled_recipe) |>
```

```
  add_model(rf_spec) |>
  fit(train_data)
```

This strategy has a clearer effect than class weighting. For random forest, fatal recall rises from 0.0684 to 0.2955, while balanced accuracy improves from 0.5233 to 0.5756. At the same time, specificity falls from 0.9782 to 0.8557, so the model starts to label more non-fatal incidents as fatal. If fatal recall is the primary criterion, oversampled XGBoost is the strongest option because it catches the largest share of fatal incidents. If the goal is a more balanced compromise, XGBoost and logistic regression end up very close on balanced accuracy. So oversampling is useful when missing fatal incidents is especially costly, but the trade-off is substantial.

This produces a more meaningful shift than weighting because it changes what the model sees during training, not just how mistakes are penalized afterward. In substantive terms, oversampling makes the models take the fatal class more seriously. The price is a visible increase in false positives. If specificity matters more, oversampled logistic regression is the most restrained of the stronger oversampled options, while random forest remains the most conservative overall. So oversampling works, but it does not by itself give the cleanest overall compromise.



This diagnostic plot shows why oversampling matters in this project. In the default models, many fatal incidents already receive somewhat higher predicted probabilities than non-fatal incidents, but a large share of them still stays below the default `0.50` cutoff. This helps explain why a model can have a reasonable ROC AUC while still missing many fatal cases at

the classification stage. After oversampling, the distributions shift to the right, so more fatal incidents cross the decision threshold. The benefit is higher fatal recall, but the cost is more false positives. That trade-off is why the later threshold discussion matters.

7.3 Threshold Moving

Threshold moving keeps the fitted probability model fixed and changes only the cutoff used to convert probabilities into class labels. I choose the cutoff on a validation split inside the training data by searching a grid of thresholds and selecting the one with the highest balanced accuracy. This is also the clearest place to handle the practical limitation of the problem: I want to improve fatal recall under class imbalance, but I also want to keep false positives within a reasonable range. In other words, the goal is not the most recall-heavy rule, but a screening-style compromise that improves fatal-case detection while preserving non-fatal specificity at a defensible level.

```r
choose_threshold <- function(fit_obj, val_data) {
  pred <- val_data |>
    bind_cols(predict(fit_obj, val_data, type = "prob"))

  map_dfr(seq(0.05, 0.95, by = 0.01), function(threshold) {
    pred_class <- factor(
      if_else(pred$.pred_fatal >= threshold, "fatal", "non_fatal"),
      levels = c("non_fatal", "fatal")
    )

    tibble(
      threshold = threshold,
      bal_accuracy = bal_accuracy_vec(pred$fatal, pred_class),
      sensitivity = sens_vec(pred$fatal, pred_class, event_level = "second"),
      specificity = spec_vec(pred$fatal, pred_class, event_level = "second")
    )
  }) |>
    arrange(desc(bal_accuracy), abs(threshold - 0.5)) |>
    slice(1)
}
```

Table 9: Selected thresholds from the validation split

model	threshold	bal_accuracy	sensitivity	specificity
Random forest	0.19	0.6238	0.7142	0.5333
XGBoost	0.23	0.6034	0.5974	0.6095

model	threshold	bal_accuracy	sensitivity	specificity
Logistic regression	0.20	0.6021	0.7010	0.5033

This first threshold table is best read as the benchmark for unconstrained threshold moving. Under a validation-based balanced-accuracy rule alone, the random forest cutoff moves to 0.19 instead of 0.50. This keeps the underlying classifier unchanged, preserves the same ROC AUC (0.6632), and improves fatal-case detection, with balanced accuracy increasing to 0.6127 and fatal recall increasing to 0.7106. But this benchmark is also quite aggressive: the recall gain comes with a very large reduction in non-fatal specificity, so it is less well aligned with the paper's final practical objective.

The main reason this unconstrained threshold-moving benchmark still matters is that the random forest already had the strongest overall ranking performance among the three primary models. Instead of rebuilding the training data, threshold moving simply uses that stronger ranking model with a more aggressive decision cutoff for an imbalanced problem. In the held-out results, that benchmark raises fatal sensitivity from 0.0684 to 0.7106, but it also reduces non-fatal specificity from 0.9782 to 0.5147. So it is useful for showing what balanced-accuracy-only threshold selection does, but it is not the final recommended operating rule for this paper.

The threshold table above uses validation-based balanced-accuracy selection alone, so the chosen cutoffs are relatively aggressive. To make that trade-off more explicit, I also re-ran threshold selection on the same 40,000-observation analysis setup with an added rule that thresholds had to be selected on the validation split under a specificity constraint of about 0.75 before choosing the best remaining cutoff. The table below then reports the held-out test performance of those preselected thresholds. This is the preferred practical operating-point analysis for the paper's final recommendation, because it shows what threshold moving looks like when false positives are constrained more directly without retuning on the test set.

```r
choose_threshold_constrained <- function(fit_obj, val_data, min_specificity = 0.75) {
  pred <- val_data |>
    bind_cols(predict(fit_obj, val_data, type = "prob"))

  map_dfr(seq(0.05, 0.95, by = 0.01), function(threshold) {
    pred_class <- factor(
      if_else(pred$.pred_fatal >= threshold, "fatal", "non_fatal"),
      levels = c("non_fatal", "fatal")
    )

    tibble(
      threshold = threshold,
      bal_accuracy = bal_accuracy_vec(pred$fatal, pred_class),
```

```
      sensitivity = sens_vec(pred$fatal, pred_class, event_level = "second"),
      specificity = spec_vec(pred$fatal, pred_class, event_level = "second")
    )
  }) |>
    filter(specificity >= min_specificity) |>
    arrange(desc(bal_accuracy), abs(threshold - 0.5)) |>
    slice(1)
}
```

Table 10: Threshold check on the 40,000-observation analysis setup: held-out test performance of thresholds selected on the validation split under a specificity constraint of about 0.75

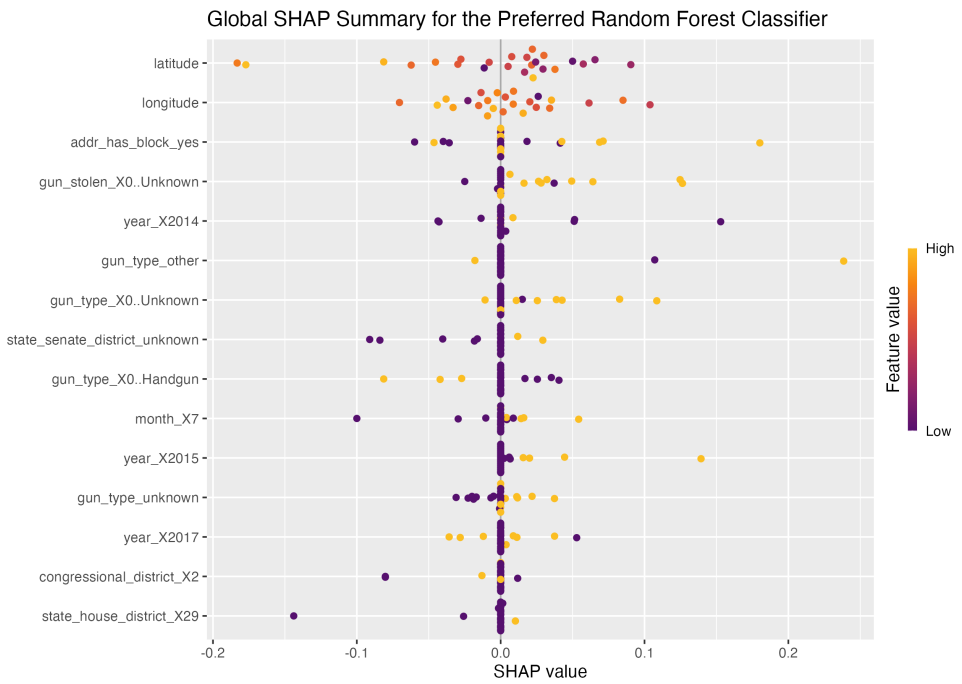
model	threshold	bal_accuracy	sensitivity	specificity
Random forest	0.27	0.602	0.462	0.743
XGBoost	0.25	0.551	0.238	0.864
Logistic regression	0.28	0.579	0.400	0.757

Under this more conservative rule, the selected thresholds move upward to about `0.25` to `0.28` rather than the more aggressive values in the main threshold table. The resulting classifiers catch fewer fatal incidents than the balanced-accuracy-optimal thresholds, but they retain meaningfully higher specificity overall and therefore better match the paper's intended screening-style use case. The thresholds are selected on the validation split under a specificity constraint of about 0.75, while the table reports held-out test performance of those preselected thresholds. Because of normal out-of-sample variation, held-out test specificity can still fall slightly below 0.75, and that is not a violation of the rule. In this constrained 40,000-sample comparison, random forest remains the strongest of the three models on balanced accuracy, while logistic regression stays reasonably competitive and XGBoost becomes much more conservative. I therefore treat this subsection as the preferred practical robustness check supporting the final recommendation. Its main message is that even after imposing a more conservative false-positive constraint during validation-stage threshold selection, random forest still looks like the strongest overall compromise among the three primary models in this 40,000-sample held-out comparison.

Putting the three strategies together, the pattern is fairly clear. Class weighting helps the least. Oversampling is more useful when recall is the dominant priority, but it is also the more aggressive option. The preferred final practical setup for this paper is specificity-constrained threshold moving on the random forest, because it starts from the strongest underlying model and then chooses a more conservative operating point that better matches the paper's screening-oriented goal. Oversampled XGBoost remains the aggressive high-recall alternative, but it is no longer the preferred overall recommendation.

8. Model Interpretation

Because the preferred final setup is the random forest under the more conservative threshold-moving rule, the interpretation should focus on the underlying random-forest classifier rather than on the threshold itself. Changing the threshold changes only the decision rule, not the fitted model. For computational feasibility, I precomputed approximate SHAP values for the same random-forest specification on a separate 4,000-observation stratified training subset used only for interpretation. The figure was generated with `fastshap` and `shapviz` after applying the same leakage-safe preprocessing steps used in the main pipeline.



The SHAP summary shows which predictors contribute most strongly to the random forest's fatal-probability predictions. The dominant variables are mainly spatial and contextual fields, including latitude/longitude, administrative location indicators, and city or county. That is substantively useful because it matches the earlier EDA: broad geographic context appears to carry real predictive signal in this dataset, even if it should be interpreted cautiously rather than causally. A second pattern is that some gun-related categories, especially unknown-style entries, also matter. One plausible interpretation is that missingness and recording style are themselves informative about the kinds of incidents that enter the file and how fully they are documented. Time markers such as year and month also contribute, which suggests that temporal context and period-specific reporting environments help separate incidents beyond any single incident attribute. These are predictive associations about how the classifier uses structured context, not claims about mechanism.

9. Discussion and Limitations

The main finding is fairly modest, but still useful. Structured incident-level features do contain predictive signal for fatal versus non-fatal outcomes, but the signal is limited and easy to overstate if evaluation focuses only on accuracy. In other words, this task is better understood as a screening problem than as a strong classification problem. Much of the usable signal also appears to be contextual, especially from location and timing variables, rather than from rich incident-specific mechanism. This is consistent with the broader literature: administrative and surveillance-style records can support predictive screening, but their usefulness depends heavily on data quality, missingness, and careful modeling choices (Goin, Rudolph, and Ahern 2018; Heller et al. 2022; Swedo et al. 2023). This pattern is especially consistent with work arguing that these kinds of records are more useful for screening than for definitive classification. In that sense, the present results are less a story of strong individual-level prediction than a story of limited but still usable contextual signal under noisy reporting conditions.

Under that task framing, the model comparison is easier to interpret. The baseline comparison starts from logistic regression as the formal benchmark, but the direct comparison shows random forest as the strongest default model in this project by ROC AUC. The imbalance analysis then shows a clearer practical distinction. Class weighting changes little. Oversampling helps detect more fatal incidents, but often with a sizable specificity cost. The preferred final setup is the random forest under the more conservative specificity-constrained threshold-moving rule, because it improves fatal-case detection relative to the default cutoff while keeping non-fatal specificity near the intended bound. Logistic regression remains a simpler and reasonably competitive benchmark, while XGBoost becomes more conservative under the same constrained-threshold rule.

This preference also reflects the practical meaning of the prediction task. In a screening-style setting, missing fatal incidents is more costly than flagging some non-fatal incidents, but false positives should not increase without limit. A false negative means the model completely misses a higher-risk case, while a false positive usually means extra attention or follow-up on a case that may not need it. For that reason, I treat fatal recall as somewhat more important than precision or raw accuracy alone, but not as the only objective. The final recommendation is therefore an operating-point choice for this dataset and use case, not a universally optimal classifier.

Several limitations remain. First, the source file contains important missingness and reporting variation, especially in detailed participant and narrative-style fields. Second, the binary fatal outcome is substantively meaningful but still coarse. Third, even the retained structured variables may reflect uneven reporting practices across places and years. Fourth, the final assessment still relies on an internal held-out test split rather than on a separate external validation dataset. A related transportability concern is especially important here: because the retained signal depends heavily on contextual and place-related features, performance may degrade when the model is transported to new regions, periods, or recording environments. Finally, the SHAP analysis is an approximate interpretation step on a smaller subset for

computational reasons, so it should be used to summarize model behavior rather than to make fine-grained substantive claims. A related substantive limitation is that the structured fields used here may capture broad context better than the immediate mechanism of fatality itself. That likely creates a ceiling on how well this kind of model can perform, even when the pipeline is carefully tuned. The main bottleneck in this project is that the strongest usable predictors are also the most context-dependent, which makes the final model more suitable for internal screening than for broad transport. With more time, the next step would be external validation on a later or separate incident file, along with a cleaner comparison of how much performance comes from broad location context versus simpler incident descriptors.

10. Conclusion

This project used a cleaned incident-level dataset to predict whether a gun violence incident would be fatal or non-fatal. The final report keeps a leakage-safe pipeline, a restrained feature set, and a direct comparison between a formal logistic-regression baseline and two selected nonlinear models on a 40,000-observation modeling sample.

The overall conclusion is cautious rather than dramatic. Structured incident-level features cannot support a strong or broadly deployable classifier here, but they do support limited screening signal for separating fatal from non-fatal incidents. Much of that usable signal appears to be contextual, especially from location and timing variables, rather than deeply incident-specific. Under that framing, random forest is the strongest underlying classifier in the direct comparison by ROC AUC, and the most defensible final operating rule is the more conservative specificity-constrained threshold-moving version of that model. This improves fatal-case detection relative to the default cutoff while keeping non-fatal specificity near the intended bound for the paper's screening-style objective. If the main priority is to catch more fatal incidents and accept more false positives, XGBoost with oversampling is the more aggressive alternative.

In this setting, clean preprocessing and an appropriate decision rule matter at least as much as the choice of classifier. A broader lesson from this project is that when the real concern is missing the most serious cases, model evaluation should not be organized around accuracy alone. A useful final system should reflect the actual cost of mistakes and choose an operating point that gives extra weight to fatal-case detection while still preserving a reasonable level of overall classification quality.

References

Goin, Dana E., Kara E. Rudolph, and Jennifer Ahern. 2018. "Predictors of Firearm Violence in Urban Communities: A Machine-Learning Approach." *Health & Place* 51: 61–67.

Heller, Sara B., Benjamin Jakubowski, Zubin Jelveh, and Max Kapustin. 2022. "Machine Learning Can Predict Shooting Victimization Well Enough to Help Prevent It." Working Paper 30170. National Bureau of Economic Research.

Polcari, Ann M. et al. 2023. "A Novel Machine-Learning Tool to Identify Community Risk for Firearm Violence." *The Journal of Trauma and Acute Care Surgery*.

Swedo, Elizabeth A. et al. 2023. "Development of a Machine Learning Model to Estimate US Firearm Homicides in Near Real Time." *JAMA Network Open* 6 (3): e233413.

Zhou, Hui, Claudia Nau, Fagen Xie, Richard Contreras, Deborah Ling Grant, Sonya Negriff, Margo Sidell, Corinna Koebnick, and Rulin Hechter. 2024. "A Machine-Learning Prediction Model to Identify Risk of Firearm Injury Using Electronic Health Records Data." *Journal of the American Medical Informatics Association* 31 (10): 2173–80.